

Source Link: https://github.com/Bharathp953/Azure/blob/main/log_reg.ipynb

```
\documentclass{article}
%\usepackage[preprint]{neurips_2024}
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{graphicx}
\usepackage{booktabs}
\usepackage{hyperref}
\usepackage{xcolor}

\title{Experimental Analysis of Loss Functions, Learning Rate Schedules, and Noise Robustness in
TensorFlow Linear Regression}

\author{
  AAM35 \\\
  Department of Computer Science \\\
  \texttt{aam35@example.com}
}

\begin{document}

\maketitle

\begin{abstract}
This report presents an experimental study of linear regression implemented using TensorFlow.
Various loss functions (MSE, Huber, Hybrid), noise types, learning rates, and adaptive scheduling
mechanisms were analyzed. The aim was to understand convergence behavior, robustness, and
reproducibility under different conditions. The findings highlight trade-offs between performance
and stability across configurations.
\end{abstract}

\section{Introduction}
```

Linear regression serves as a foundation for understanding optimization in deep learning. This work explores how loss functions, learning rate schedules, and noise affect convergence, stability, and generalization.

\section{Methodology}

We implemented a linear regression model defined as:

$$\hat{y} = Wx + b$$

with trainable parameters W and b . Data was generated synthetically as:

$$y = 3x + 2 + \epsilon,$$

where ϵ represents random noise. Training was performed using TensorFlow 2.x for 1000 steps with an initial learning rate of 0.01.

\section{Loss Functions}

\subsection{Mean Squared Error (MSE)}

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MSE punishes large errors more severely and is ideal for Gaussian noise but sensitive to outliers.

\subsection{Huber Loss}

$$L_{\text{Huber}} = \begin{cases} \frac{1}{2} (y_i - \hat{y}_i)^2, & |y_i - \hat{y}_i| \leq \delta \\ \delta (|y_i - \hat{y}_i| - \frac{1}{2} \delta), & \text{otherwise} \end{cases}$$

Combines L_1 and L_2 behavior, reducing sensitivity to outliers. Works better in the presence of large deviations.

`\subsection{Hybrid Loss (L1 + L2)}`

`\[`

`L_{\text{Hybrid}} = \alpha |y - \hat{y}| + (1-\alpha)(y - \hat{y})^2`

`\]`

Balances robustness and smooth convergence. Tunable via α .

`\section{Learning Rate and Scheduling}`

Learning rates tested:

`\begin{itemize}`

`\item  $0.1$  – fast convergence but oscillatory.`

`\item  $0.001$  – slower but stable.`

`\item  $0.01$  – balanced (default).`

`\end{itemize}`

A patience scheduler halves the learning rate if loss stagnates for 10 epochs:

`\[`

`\eta_{t+1} =`

`\begin{cases}`

`\eta_t / 2, & \text{if no improvement for } p \text{ epochs} \\`

`\eta_t, & \text{otherwise}`

`\end{cases}`

`\]`

`\section{Experiments}`

`\subsection{Initialization Effects}`

Starting with $W=5.0$, $b=-5.0$ still converged to $W \approx 3.0$, $b \approx 2.0$, but required more steps.

`\subsection{Noise Experiments}`

We examined three types of noise:

```
\begin{itemize}
  \item Gaussian (standard deviation 0.5)
  \item Uniform ( $[-2, 2]$ )
  \item Salt-and-pepper (occasional spikes)
\end{itemize}
```

Huber and Hybrid losses were more stable under non-Gaussian and high-variance noise.

\subsection{Noise in Weights and Learning Rate}

Small perturbations were added each epoch:

```
\[
W \leftarrow W + \mathcal{N}(0, 0.01), \quad
\eta \leftarrow \eta_0(1 + \mathcal{N}(0, 0.1))
\]
```

This stochasticity improved generalization, acting like regularization.

\section{Performance Effects Summary}

```
\begin{itemize}
  \item \textbf{MSE:} Fast convergence, poor robustness to outliers; best for Gaussian noise.
  \item \textbf{Huber:} Moderate convergence, high robustness; best balance overall.
  \item \textbf{L1:} Slow convergence, very high robustness; more stable.
  \item \textbf{High Noise:} Slower learning, reduced accuracy; needs robust loss.
  \item \textbf{Patience Scheduling:} Stabilizes learning and helps reach better minima.
  \item \textbf{Weight/Learning-rate Noise:} Adds stochasticity, improves generalization.
  \item \textbf{Bad Initialization:} Slower or diverging; use small randoms.
  \item \textbf{Small Learning Rate:} Stable but slow; good for noisy data.
\end{itemize}
```

\section{Repeatability and Seeds}

Results vary across runs due to random initialization and GPU non-determinism.

Setting a fixed seed ensures reproducibility:

\[

\text{Seed}_{\text{AAM}} = 65 + 65 + 77 = 207

\]

Thus, `tf.random.set_seed(207)` guarantees consistent results.

\section{GPU vs CPU Timing}

For small models, GPU $\approx$ CPU in runtime, but GPU outperforms for larger datasets or batch computations.

\section{Robustness and Noise Benefits}

Adding controlled noise acts as regularization, preventing overfitting and improving generalization.

However, excessive noise prevents convergence.

\section{Final Report Summary}

\begin{itemize}

\item \textbf{Baseline (MSE):} Normal Gaussian noise; converged to $W=3.02$, $b=2.01$ — performs well under clean conditions.

\item \textbf{Huber Loss:} Added outlier noise; stable convergence — robust to outliers.

\item \textbf{Hybrid Loss:} Mixed noise; balanced performance — good trade-off.

\item \textbf{Patience Scheduling:} Adaptive learning rate — smoother convergence and improved stability.

\item \textbf{Noise in Data:} Increased variance slows learning — needs robust loss.

\item \textbf{Noise in Weights:} Small random perturbations — improves generalization, acts as regularization.

\item \textbf{Learning Rate Noise:} $\pm 10\%$ variation — slightly noisy but stable convergence.

\end{itemize}

\section{Conclusion}

The experiments show that MSE excels under Gaussian noise, while Huber and Hybrid losses provide robustness under noisy or outlier-heavy data. Adaptive learning rate scheduling and mild stochastic noise improve convergence and generalization. Proper initialization and reproducibility controls are essential for consistent training outcomes.

\end{document}